CÓDIGOCLARO · RUTA JAVA

Tema 13 · Colecciones

Cuaderno de estudio para DAM/DAW · 10 lecciones

Qué vas a aprender

List, Set, Map, ArrayList, HashSet y HashMap.

Este cuaderno trabaja colecciones en pasos cortos. La meta no es repetir palabras de memoria: es reconocer el problema, elegir una herramienta adecuada, explicar por qué funciona y comprobarla con datos distintos. Reserva una sesión para leer el modelo, otra para resolver el ejercicio y una tercera para intentar el laboratorio sin mirar la solución.

Resultados de aprendizaje

- Explicar con tus palabras qué problema resuelve colecciones y en qué parte de una aplicación DAM/DAW aparece.
- Leer el ejemplo, predecir el resultado y justificar cada decisión relevante.
- Resolver una práctica, comprobar casos normales y límite, y depurar un fallo con un método reproducible.

Cómo estudiar este tema

Ten a mano un editor compatible con la tecnología y copia los ejemplos en un archivo nuevo. Antes de ejecutar, escribe tu predicción. Si no coincide con la salida, no borres el intento: anota qué supuesto era incorrecto y prueba una variación mínima. Al terminar, explica el concepto a otra persona usando un ejemplo propio.

Punto de control inicial

Antes de avanzar, escribe dos ideas que ya conoces sobre este tema y una pregunta que todavía no sabes responder. Vuelve a ellas al final de la lección 10. Si cambias de respuesta, explica qué ejemplo o prueba hizo que cambiaras de opinión.

Lección 1 · El modelo mental

Idea central

List conserva orden y admite duplicados; Set representa elementos únicos; Map relaciona claves y valores. ArrayList y HashSet son implementaciones habituales. Programa contra interfaces; no confíes en el orden de HashSet o HashMap. En ArrayList, remove(int) elimina por índice y remove(Object) elimina la primera coincidencia. Comparable define el orden natural; TreeMap ordena sus claves con esa comparación y considera una clave equivalente a otra cuando compareTo devuelve cero. Añade un desempate si varias claves pueden compartir el criterio principal. En colecciones hash, equals() y hashCode() deben respetar su contrato.

Para razonar sobre colecciones, separa cuatro preguntas: qué datos existen al empezar, qué regla se aplica, qué cambia durante el proceso y qué resultado debe observar quien usa el programa. Esta separación ayuda a no confundir la sintaxis con la finalidad. Una línea puede compilar y aun así resolver mal el problema si el dato representa otra cosa o falta un caso.

En Java, ten presente este marco

En este itinerario se trabaja Java con tipos estáticos y ejecución sobre la JVM. El compilador comprueba muchos errores de tipos antes de ejecutar y genera bytecode; la JVM lo carga y ejecuta. Distingue los ocho tipos primitivos de las referencias a objetos. Para texto, compara el contenido con equals(), no la identidad con ==. Las conversiones numéricas pueden perder precisión o desbordar si no se elige el tipo adecuado.

Comprueba tu comprensión

- ¿Qué parte de la explicación define la regla principal de colecciones?
- ¿Qué dato puede cambiar y qué dato debería permanecer estable?
- ¿Qué observación concreta demostraría que tu programa cumple el objetivo?

Lección 2 · Vocabulario y reglas

Reglas de Java que afectan a este tema

ArrayList mantiene orden y admite repetidos; HashSet modela pertenencia sin duplicados; HashMap relaciona claves únicas con valores. List, Set y Map describen contratos, mientras las clases concretas aportan implementaciones. No cambies una colección estructuralmente mientras la recorres con un iterador salvo que uses la operación admitida por ese iterador.

Ideas específicas de colecciones

Elige la estructura de datos según la operación que el programa necesite: una secuencia conserva posición, un conjunto representa pertenencia sin duplicados y un mapa asocia claves con valores. Distingue acceso por índice de acceso por clave y observa si la estructura es mutable. Si dos variables apuntan al mismo objeto mutable, cambiarlo desde una puede afectar a la otra.

Glosario de trabajo

Término	Significado en este recorrido
Secuencia	Estructura ordenada a la que se suele acceder por posición.
Clave	Dato usado para localizar un valor en un mapa o diccionario.
Mutabilidad	Posibilidad de cambiar un objeto después de crearlo.
Duplicado	Elemento repetido que una estructura puede conservar o descartar.

Relaciona cada término con una línea del ejemplo o una decisión del ejercicio. Comprueba las diferencias de versión en la fuente oficial enlazada antes de aplicar una regla al proyecto.

Lección 3 · Lectura guiada del ejemplo

Primero identifica la entrada, luego sigue el valor por el programa. No supongas que una asignación transforma el dato: a veces solo enlaza un nombre con el mismo objeto. Observa dónde empieza cada bloque, qué condición activa una rama y qué instrucción produce la salida. Si el ejemplo tiene varias líneas, ejecútalo por partes y añade temporalmente observaciones seguras para ver los valores intermedios.

Código · java

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        List<String> nombres = new ArrayList<>(List.of("Luz","Paz","Luz"));
        Set<String> unicos = new HashSet<>(nombres);
        Map<String,Integer> cuenta = new HashMap<>();
        for (String nombre : nombres) cuenta.merge(nombre, 1, Integer::sum);
        System.out.println(nombres.size() + ", " + unicos.size());
        System.out.println(cuenta.get("Luz"));
    }
}
```

Salida esperada

```
3, 2
2
```

Registro de la traza

Momento	Qué anotar
Antes de ejecutar	Valores iniciales, sus tipos y qué operación esperas que ocurra.
Después de cada paso	Nombre afectado, valor nuevo y regla que produjo el cambio.
Al encontrar una decisión	Condición evaluada y camino que se ejecutó.
Al terminar	Salida real, diferencia respecto a tu predicción y causa.
Con una entrada distinta	Qué resultado cambia y qué parte del contrato permanece igual.

Traza en papel

- Antes de la primera instrucción: apunta los valores iniciales y sus tipos.
- Después de cada operación: anota solo los nombres que hayan cambiado y la razón.
- Al final: compara tu predicción con «3, 2 2» y explica la diferencia si la hay.

Cambia un dato de entrada y vuelve a hacer la traza. Una buena explicación debe anticipar el nuevo resultado, no limitarse a repetir la salida original.

Lección 4 · Decidir cuándo usarlo

Elige la estructura de datos según la operación que el programa necesite: una secuencia conserva posición, un conjunto representa pertenencia sin duplicados y un mapa asocia claves con valores. Distingue acceso por índice de acceso por clave y observa si la estructura es mutable. Si dos variables apuntan al mismo objeto mutable, cambiarlo desde una puede afectar a la otra.

Caso de uso

Un inventario necesita eliminar por posición o por valor y presentar productos primero por precio y luego por nombre.

Compara antes de elegir

- ¿La solución más directa deja claro el propósito o requiere conocer demasiados detalles internos?
- ¿Qué coste aparece si crece el volumen de datos o se añade un nuevo caso?
- ¿Qué comprobación automatizada detectaría una modificación accidental?

Enfoques relacionados

- Usar Comparator.comparingInt(...).thenComparing(...) y pasarlo a TreeMap cuando el orden pertenezca a la colección y no al tipo.
- Usar TreeMap<String, Integer> para frecuencias cuando las claves de texto deban imprimirse ordenadas.

No existe una alternativa mejor en todos los contextos. Elige una y escribe su ventaja y su límite para el tamaño, la versión y los requisitos del ejercicio. Si dos enfoques cumplen exactamente el mismo contrato y ambos son legibles, pueden considerarse válidos.

Lección 5 · Errores y diagnóstico

Fallos que conviene reconocer

- Esperar que List rechace duplicados. Para localizarlo, reduce el caso a la menor entrada que lo reproduce, observa el valor implicado y verifica la regla antes de cambiar el código.

- Suponer que HashMap conserva el orden de inserción. Para localizarlo, reduce el caso a la menor entrada que lo reproduce, observa el valor implicado y verifica la regla antes de cambiar el código.
- Confundir remove(2), que elimina el índice 2 en ArrayList<Integer>, con remove(Integer.valueOf(2)), que elimina el primer valor 2. Para localizarlo, reduce el caso a la menor entrada que lo reproduce, observa el valor implicado y verifica la regla antes de cambiar el código.
- Sobrescribir equals sin mantener el contrato de hashCode en claves de HashMap o elementos de HashSet. Para localizarlo, reduce el caso a la menor entrada que lo reproduce, observa el valor implicado y verifica la regla antes de cambiar el código.
- Ordenar productos solo por precio y devolver cero para nombres distintos; TreeMap los trataría como la misma clave. Para localizarlo, reduce el caso a la menor entrada que lo reproduce, observa el valor implicado y verifica la regla antes de cambiar el código.

Rutina de depuración

1. Describe el resultado esperado y el resultado real sin mezclar ambos. 2. Reproduce el fallo con una entrada fija. 3. Reduce el programa hasta dejar la primera operación que se comporta distinto. 4. Formula una hipótesis comprobable y cambia una sola cosa. 5. Añade una prueba que impida que el mismo fallo vuelva. Este orden también funciona cuando el error aparece en una API, una interfaz o un archivo.

Preguntas de revisión

- ¿El error está en los datos, en la regla o en cómo se presenta el resultado?
- ¿Qué caso límite no estaba cubierto por el ejemplo inicial?
- ¿La solución corrige la causa o solo oculta el síntoma?

Lección 6 · Práctica guiada

Situación de DAM/DAW

Un inventario necesita eliminar por posición o por valor y presentar productos primero por precio y luego por nombre.

Enunciado

Muestra en dos copias de una ArrayList la diferencia entre remove(2) y remove(Integer.valueOf(2)). Después implementa Producto como Comparable, con orden natural por precio y desempate por nombre, y guarda productos en un TreeMap. Incluye una inserción repetida de la misma clave.

Convierte el enunciado en requisitos verificables

- Escribe qué entradas acepta la solución y qué debe devolver o mostrar.
- Enumera las reglas de negocio y separa las que son obligatorias de las decisiones de presentación.
- Antes de programar, completa la tabla de pruebas y predice cada resultado.

Casos que debe superar

Entrada	Resultado esperado
valores=[7,2,9,2]; aplicar remove(2) y remove(Integer.valueOf(2)) en copias separadas	remove(2) => [7, 2, 2]; remove(Integer.valueOf(2)) => [7, 9, 2]
TreeMap con (B,5)=1, (A,5)=2 y (C,3)=1	Orden natural: C:3=1, A:5=2, B:5=1; el desempate por nombre conserva A y B.
Insertar (A,5)=2 y luego otra vez (A,5)=4	Primero C:3=1, A:5=2, B:5=1; tras insertar (A,5)=4: C:3=1, A:5=4, B:5=1. La clave equivalente actualiza su valor.

Qué demuestra cada caso

Clasifica cada fila de la tabla como caso habitual, límite o variación importante. Si el enunciado no define qué hacer con una entrada fuera de contrato, no inventes un resultado silencioso: anota la ambigüedad y propone una regla antes de programar. En la práctica profesional, acordar esa regla evita que dos implementaciones parezcan correctas y se comporten de forma incompatible.

Plan de solución

- Divide la tarea en pasos pequeños que puedan comprobarse por separado.
- Implementa primero el caso habitual y después añade los límites y errores previstos.
- Ejecuta cada caso tras un cambio; no esperes al final para descubrir qué parte falló.

Lección 7 · Solución explicada

Una solución no se justifica solo porque la salida coincida en un ejemplo. Debe respetar el contrato para todas las entradas admitidas. Lee el código siguiente y vincula cada operación con una frase del enunciado. Si no puedes señalar esa relación, revisa el diseño antes de continuar.

Código · java

```
import java.util.*;
record Producto(String nombre, int precio) implements Comparable<Producto> {
    @Override public int compareTo(Producto otro) {
        int c = Integer.compare(precio, otro.precio);
        return c != 0 ? c : nombre.compareTo(otro.nombre);
    }
}
public class Main {
    public static void main(String[] args) {
        var porIndice = new ArrayList<>(List.of(7, 2, 9, 2));
        var porValor = new ArrayList<>(porIndice);
        porIndice.remove(2);
        porValor.remove(Integer.valueOf(2));
        System.out.println(porIndice + " / " + porValor);
        var mapa = new TreeMap<Producto, Integer>();
        mapa.put(new Producto("B", 5), 1); mapa.put(new Producto("A", 5), 2);
        mapa.put(new Producto("C", 3), 1); mapa.forEach((p, n) -> System.out.
println(p.nombre() + ":" + p.precio() + "=" + n));
        mapa.put(new Producto("A", 5), 4); mapa.forEach((p, n) -> System.out.
println(p.nombre() + ":" + p.precio() + "=" + n));
    }
}
```

Por qué se sigue este razonamiento

remove(2) selecciona el tercer elemento (9), mientras que remove(Integer.valueOf(2)) busca y elimina la primera coincidencia. Producto compara precio y, si empata, nombre; por eso A y B se conservan y C aparece primero. La segunda inserción de A con el mismo precio compara como cero y reemplaza su valor en TreeMap. Los resultados de los tres casos se pueden comprobar directamente.

Comprobación de corrección

Comprueba cada caso de la tabla y explica qué parte de la solución lo cubre. Después revisa el caso mínimo, el máximo razonable y una entrada inválida si el contrato la permite. Comprueba también los tipos de entrada y salida: una coincidencia textual puede ocultar un valor mal tipado o una conversión con pérdida.

Paso	Justificación que debes poder dar
Contrato	Qué valores acepta el ejercicio y qué resultado promete.
Regla	Qué operación transforma la entrada en la salida.
Pruebas	Qué caso comprueba lo normal y cuál descubre un límite.

Alternativas válidas

- Usar Comparator.comparingInt(...).thenComparing(...) y pasarlo a TreeMap cuando el orden pertenezca a la colección y no al tipo.
- Usar TreeMap<String, Integer> para frecuencias cuando las claves de texto deban imprimirse ordenadas.

Al comparar soluciones, prioriza contrato correcto, legibilidad y pruebas. Una alternativa que cambia la representación interna puede ser válida; una que cambia silenciosamente el significado de la salida no lo es.

Lección 8 · Laboratorio de aplicación

Retoma el ejercicio de colecciones y amplíalo como una pequeña funcionalidad de una aplicación de gestión académica. Conserva el requisito original y añade una regla nueva que tenga sentido para los datos. Antes de modificar el código, escribe la regla en lenguaje natural, un ejemplo que la cumpla y otro que la incumpla.

Variación para transferir lo aprendido

Prueba dos productos con nombres distintos y el mismo precio, y después dos objetos con el mismo nombre y precio. Comprueba el orden y explica cuándo TreeMap reemplaza el valor de una clave equivalente.

Desafíos progresivos

- Nivel 1: añade un caso límite que no aparecía en la tabla y registra el resultado esperado.
- Nivel 2: cambia la forma de recibir o recorrer los datos sin cambiar el contrato externo.
- Nivel 3: separa la regla principal de su entrada/salida y prueba ambas partes de forma independiente.

Criterios de entrega

- La solución conserva el resultado esperado del ejercicio original.
- El nuevo comportamiento queda cubierto por una prueba que falla antes del cambio.
- Los nombres permiten seguir el flujo sin comentarios que repitan la sintaxis.
- La entrada inválida o el error externo tiene una respuesta definida.

Prueba de regresión

Guarda los casos originales antes de añadir la regla nueva. Ejecuta la batería completa después de cada cambio, no solo el nuevo caso. Si una prueba deja de pasar, decide si encontraste un defecto o si cambió el contrato y hay que actualizar la expectativa con una explicación. Termina con una nota breve: qué cambió, qué comprobaste y qué límite sigue abierto.

Anota qué parte se puede reutilizar en otro módulo y qué parte depende del contexto de la aplicación. Esa distinción ayuda a decidir si merece la pena extraer una función o componente.

Lección 9 · Síntesis y fuentes

Resumen del tema

Has estudiado colecciones desde el modelo conceptual hasta una práctica comprobable. La secuencia de trabajo es: identificar entradas y contrato, elegir la operación adecuada, seguir el flujo, comprobar casos límite y conservar una prueba para la siguiente modificación. Aplica las reglas de Java sin confundir una característica del lenguaje con una API concreta del entorno.

Lista de control antes de avanzar

- Puedo explicar el concepto con un ejemplo propio y predecir su salida.
- Puedo identificar al menos un caso límite y un error habitual.
- Puedo comparar dos enfoques y justificar mi elección.
- Puedo verificar una solución sin depender de mirar el resultado una sola vez.

Fuentes oficiales para ampliar

Dev.java — The Collections Framework · Oracle — Java Collections Framework · Oracle — Object.equals y hashCode, Java SE 25 · Oracle / ArrayList.remove(int) y remove(Object), Java SE 25 · Oracle / Comparable, Java SE 25 · Oracle / TreeMap, Java SE 25

Consulta la sección de la fuente que corresponda a la versión de tu proyecto. Este cuaderno resume conceptos para estudiar; cuando haya una diferencia de detalle, prevalece la documentación oficial del lenguaje o biblioteca.

Lección 10 · Repaso activo

Cierra el material y responde sin copiar frases. Si dudas, vuelve a la página indicada, comprueba la fuente primaria y reescribe la respuesta con tus palabras. La recuperación activa ayuda a detectar qué parte aún depende de memorizar un ejemplo concreto.

Preguntas

- ¿Qué problema aborda colecciones y qué dato necesita?
- ¿Qué diferencia hay entre una solución que funciona en el ejemplo y una que cumple el contrato?
- ¿Qué error frecuente te parece más probable y cómo lo reproducirías?
- ¿Qué alternativa elegirías en un proyecto pequeño y por qué?

Respuestas de comprobación

- La idea clave se resume en esta explicación: List conserva orden y admite duplicados; Set representa elementos únicos; Map relaciona claves y valores. ArrayList y HashSet son implementaciones habituales. Programa contra interfaces; no confíes en el orden de HashSet o HashMap. En ArrayList, remove(int) elimina por índice y remove(Object) elimina la primera coincidencia. Comparable define el orden natural; TreeMap ordena sus claves con esa comparación y considera una clave equivalente a otra cuando compareTo devuelve cero. Añade un desempate si varias claves pueden compartir el criterio principal. En colecciones hash, equals() y hashCode() deben respetar su contrato.
- Una fuente inicial de fallo descrita en el tema es: Esperar que List rechace duplicados.
- Una alternativa es válida si mantiene las entradas, salidas y reglas acordadas y supera las mismas pruebas.